Yes. Below is a **clean, Word-ready offline coding guide** for the File module. You can copy everything from here into Word and work through it step by step without needing internet access or asking me what comes next.

FILE MODULE — COMPLETE CODING BREAKDOWN

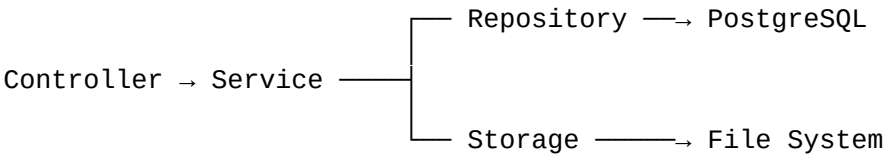
Project: Project Management System

Development style

We continue using the architecture already established in your backend:

```
Controller
    ↓
Service
    ↓
Repository
    ↓
Prisma / PostgreSQL
```

For files, we add one additional layer:



The File module must also cooperate with:

```
FileService
    ├── Authorization
    ├── ActivityService
    └── StorageService
```

PART 1 — WHAT WE ARE BUILDING

The File module will allow users to:

1. Upload files.
2. Attach files to Tasks.
3. Attach files to Projects.
4. Attach files to Workspaces.
5. Attach files to Comments.
6. View file metadata.

7. Download files.
8. Delete files.
9. List files belonging to a Task.
10. List files belonging to a Project.
11. List files belonging to a Workspace.
12. List files belonging to a Comment.
13. Record file activities.

For the first implementation we use:

```
Local File System
```

Later we can replace it with:

```
Supabase Storage
AWS S3
Cloudinary
etc.
```

without redesigning the whole File module.

PART 2 — IMPORTANT CONCEPT

A file has **two different parts**.

Part A — Physical file

Example:

```
uploads/files/
    5f7a1c23-....pdf
```

This is the actual file.

Part B — Database information

PostgreSQL stores:

```
id
originalName
storedName
mimeType
size
storageKey
uploadedById
taskId
projectId
workspaceId
commentId
createdAt
updatedAt
```

So:

```
Actual file
     ↓
Storage

File information
     ↓
PostgreSQL
```

The database does **not** store the actual PDF/image/video bytes in our first implementation.

PART 3 — FINAL FOLDER STRUCTURE

Create this structure:

```
project-root/
│
├── src/
│   │
│   ├── config/
│   │   ├── prisma.ts
│   │   ├── env.ts
│   │   └── multer.ts
│   │
│   ├── infrastructure/
│   │   │
│   │   └── storage/
│   │       ├── storage.types.ts
│   │       └── local.storage.ts
│   │
│   ├── middleware/
│   │   └── auth.middleware.ts
│   │
│   └── modules/
│       │
│       ├── user/
│       ├── workspace/
│       ├── project/
│       ├── task/
│       ├── comment/
│       ├── notification/
│       ├── activity/
│       │
│       └── file/
│           ├── file.controller.ts
│           ├── file.service.ts
│           ├── file.repository.ts
│           ├── file.routes.ts
│           ├── file.types.ts
│           └── file.validation.ts
│
├── uploads/
│   └── files/
│
```

```
├── prisma/
│   └── schema.prisma
│
└── package.json
```

PART 4 — INSTALL REQUIRED PACKAGE

You only need Multer for the first implementation.

Run:

```
npm install multer
```

Then:

```
npm install -D @types/multer
```

Check:

```
npm list multer
```

Expected:

```
multer
```

Do not install cloud-storage packages yet.

PART 5 — UPDATE PRISMA SCHEMA

Add the following model.

```
model File {
  id           String   @id @default(uuid())

  originalName String
  storedName   String
  mimeType     String
  size         Int
  storageKey   String

  uploadedById String
  uploadedBy   User @relation(
    fields: [uploadedById],
    references: [id]
  )

  workspaceId String?
  workspace   Workspace? @relation(
    fields: [workspaceId],
    references: [id]
```

```
  )

  projectId String?
  project   Project? @relation(
    fields: [projectId],
    references: [id]
  )

  taskId String?
  task Task? @relation(
    fields: [taskId],
    references: [id]
  )

  commentId String?
  comment Comment? @relation(
    fields: [commentId],
    references: [id]
  )

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([uploadedById])
  @@index([workspaceId])
  @@index([projectId])
  @@index([taskId])
  @@index([commentId])

  @@map("files")
}
```

PART 6 — ADD USER RELATION

Inside `User` add:

```
uploadedFiles File[]
```

Your User will conceptually contain:

```
model User {

  // existing fields...

  uploadedFiles File[]

}
```

Meaning:

```
One User
    ↓
Many Files
```

PART 7 — ADD WORKSPACE RELATION

Inside `Workspace`:

```
files File[]
```

Meaning:

```
One Workspace
    ↓
Many Files
```

PART 8 — ADD PROJECT RELATION

Inside `Project`:

```
files File[]
```

PART 9 — ADD TASK RELATION

Inside `Task`:

```
files File[]
```

PART 10 — ADD COMMENT RELATION

Inside `Comment`:

```
files File[]
```

PART 11 — CHECK PRISMA

Run:

```
npx prisma format
```

Then:

```
npx prisma validate
```

If there are no errors:

```
npx prisma migrate dev --name add_file_module
```

Then:

```
npx prisma generate
```

Do not continue until Prisma validates successfully.

PART 12 — UNDERSTAND THE FILE FIELDS

`originalName`

The name the user uploaded.

Example:

```
database-design.pdf
```

`storedName`

The name we physically store.

Example:

```
2b5f4b2c-9c7d-4e....pdf
```

We generate this ourselves.

`mimeType`

Example:

```
application/pdf
image/png
image/jpeg
text/plain
```

---

size

File size in bytes.

Example:

`245678`

---

storageKey

The location/key used by our storage layer.

Example:

`files/2b5f4b2c-9c7d-4e....pdf`

---

uploadedById

The authenticated user who uploaded the file.

---

taskId

Optional relationship to a Task.

---

projectId

Optional relationship to a Project.

---

workspaceId

Optional relationship to a Workspace.

---

commentId

Optional relationship to a Comment.

PART 13 — FILE TYPES

Create:

`src/modules/file/file.types.ts`

Add:

```
export interface CreateFileInput {
  originalName: string;
  storedName: string;
  mimeType: string;
  size: number;
  storageKey: string;

  uploadedById: string;

  workspaceId?: string;
  projectId?: string;
  taskId?: string;
  commentId?: string;
}
```

Also:

```
export interface FileAttachmentInput {
  workspaceId?: string;
  projectId?: string;
  taskId?: string;
  commentId?: string;
}
```

PART 14 — FILE REPOSITORY

Create:

`src/modules/file/file.repository.ts`

Import Prisma:

```
import { prisma } from "../../config/prisma.js";
import type { CreateFileInput } from "./file.types.js";
```

Create class:

```
export class FileRepository {

}
```

PART 15 — CREATE FILE

Add:

```
async create(data: CreateFileInput) {
  return await prisma.file.create({
    data: {
      originalName: data.originalName,
      storedName: data.storedName,
      mimeType: data.mimeType,
      size: data.size,
      storageKey: data.storageKey,

      uploadedById: data.uploadedById,

      workspaceId: data.workspaceId,
      projectId: data.projectId,
      taskId: data.taskId,
      commentId: data.commentId,
    },
  });
}
```

PART 16 — FIND FILE BY ID

Add:

```
async findById(id: string) {
  return await prisma.file.findUnique({
    where: {
      id,
    },
  });
}
```

PART 17 — FIND TASK FILES

Add:

```
async findByTask(taskId: string) {
  return await prisma.file.findMany({
    where: {
      taskId,
    },
    orderBy: {
      createdAt: "desc",
    },
  });
}
```

PART 18 — FIND PROJECT FILES

```
async findByProject(projectId: string) {
  return await prisma.file.findMany({
    where: {
      projectId,
    },
    orderBy: {
      createdAt: "desc",
    },
  });
}
```

PART 19 — FIND WORKSPACE FILES

```
async findByWorkspace(workspaceId: string) {
  return await prisma.file.findMany({
    where: {
      workspaceId,
    },
    orderBy: {
      createdAt: "desc",
    },
  });
}
```

PART 20 — FIND COMMENT FILES

```
async findByComment(commentId: string) {
  return await prisma.file.findMany({
    where: {
      commentId,
    },
    orderBy: {
      createdAt: "desc",
    },
  });
}
```

PART 21 — DELETE FILE DATABASE RECORD

```
async delete(id: string) {
  return await prisma.file.delete({
    where: {
      id,
    },
  });
}
```

PART 22 — STORAGE ABSTRACTION

Create:

`src/infrastructure/storage/storage.types.ts`

Add:

```
export interface StoredFile {
  storedName: string;
  storageKey: string;
}
```

Then:

```
export interface StorageService {
  save(
    file: Express.Multer.File
  ): Promise<StoredFile>;

  delete(
    storageKey: string
  ): Promise<void>;
}
```

PART 23 — WHY STORAGE SERVICE EXISTS

Do not make FileService directly use:

`fs.writeFile()`

Instead:

```
FileService
      ↓
StorageService
      ↓
LocalStorage
      ↓
File System
```

Later:

```
FileService
      ↓
StorageService
      ↓
SupabaseStorage
```

This means the FileService doesn't care where the file is stored.

PART 24 — LOCAL STORAGE

Create:

```
src/infrastructure/storage/local.storage.ts
```

Imports:

```
import fs from "node:fs/promises";
import path from "node:path";
import crypto from "node:crypto";

import type {
  StorageService,
  StoredFile,
} from "./storage.types.js";
```

Create:

```
export class LocalStorage
  implements StorageService {

}
```

PART 25 — STORAGE DIRECTORY

Inside the class:

```
private readonly uploadDirectory =
  path.resolve("uploads/files");
```

PART 26 — SAVE FILE

Implement:

```
async save(
```

```
  file: Express.Multer.File,
): Promise<StoredFile> {

  await fs.mkdir(
    this.uploadDirectory,
    {
      recursive: true,
    },
  );

  const extension =
    path.extname(file.originalname);

  const storedName =
    `${crypto.randomUUID()}${extension}`;

  const filePath =
    path.join(
      this.uploadDirectory,
      storedName,
    );

  await fs.writeFile(
    filePath,
    file.buffer,
  );

  return {
    storedName,
    storageKey: `files/${storedName}`,
  };
}
```

PART 27 — DELETE PHYSICAL FILE

Add:

```
async delete(
  storageKey: string,
): Promise<void> {

  const fileName =
    path.basename(storageKey);

  const filePath =
    path.join(
      this.uploadDirectory,
      fileName,
    );

  await fs.unlink(filePath);
}
```

PART 28 — WHY `path.basename()`?

Never allow a user-controlled storage key to directly become a filesystem path.

We want:

```
files/file.pdf
```

to become:

```
file.pdf
```

and then:

```
uploads/files/file.pdf
```

This prevents arbitrary filesystem path manipulation.

PART 29 — MULTER CONFIGURATION

Create:

```
src/config/multer.ts
```

Add:

```
import multer from "multer";

const storage =
  multer.memoryStorage();

export const upload = multer({
  storage,

  limits: {
    fileSize:
      20 * 1024 * 1024,
  },
});
```

Maximum:

```
20 MB
```

You can change this later.

PART 30 — WHY MEMORY STORAGE?

Multer receives:

```
HTTP multipart/form-data
```

and creates:

```
req.file
```

with:

```
originalname
mimetype
size
buffer
```

We then send the buffer to:

```
LocalStorage
```

Flow:

```
Client
 ↓
Multer
 ↓
Buffer
 ↓
LocalStorage
 ↓
Disk
```

PART 31 — FILE VALIDATION

Create:

```
src/modules/file/file.validation.ts
```

Start with:

```
export const allowedMimeTypes = [
  "application/pdf",
  "image/jpeg",
  "image/png",
  "image/webp",
  "text/plain",
];
```

Then:

```
export function isAllowedMimeType(
  mimeType: string,
): boolean {
  return allowedMimeTypes.includes(
    mimeType,
  );
}
```

PART 32 — FILE SERVICE

Create:

```
src/modules/file/file.service.ts
```

Import:

```
import { FileRepository } from "./file.repository.js";
import { LocalStorage } from "../../infrastructure/storage/local.storage.js";
import {
  isAllowedMimeType,
} from "./file.validation.js";
```

Create:

```
const fileRepository =
  new FileRepository();

const storageService =
  new LocalStorage();
```

Then:

```
export class FileService {

}
```

PART 33 — FIRST SERVICE FUNCTION

Create:

```
async uploadTaskFile(
  file: Express.Multer.File,
  userId: string,
  taskId: string,
) {

}
```

The responsibility of this function is:

```
1. Validate file
2. Check Task
3. Check authorization
4. Save physical file
5. Save DB record
6. Clean up if DB fails
7. Create activity
8. Return result
```

PART 34 — VALIDATE FILE

Inside service:

```
if (!file) {
  throw new Error(
    "File is required",
  );
}
```

Then:

```
if (
  !isAllowedMimeType(
    file.mimetype,
  )
) {
  throw new Error(
    "File type is not allowed",
  );
}
```

PART 35 — CHECK FILE SIZE

```
if (file.size <= 0) {
  throw new Error(
    "File cannot be empty",
  );
}
```

Multer already handles the configured maximum size.

PART 36 — CHECK TASK

Use your existing TaskService/repository according to your current architecture.

Conceptually:

```
const task =
  await taskService.getTask(taskId);
```

Then:

```
if (!task) {
  throw new Error(
    "Task not found",
  );
```

```
}
```

Do not create a File referencing a nonexistent Task.

PART 37 — AUTHORIZATION

You must check:

```
Can this user upload to this Task?
```

The relationship is:

```
User
 ↓
Workspace membership
 ↓
Workspace
 ↓
Project
 ↓
Task
```

Your previously established rule was:

```
Workspace members can work on
projects in that workspace.
```

So your authorization logic must respect that rule.

Do not simply check:

```
userId === task.creatorId
```

because the creator is not necessarily the only person allowed to work with the task.

PART 38 — SAVE PHYSICAL FILE

After authorization:

```
const storedFile =
  await storageService.save(file);
```

You receive:

```
{
  storedName,
  storageKey
}
```

PART 39 — SAVE DATABASE RECORD

Then:

```
const createdFile =
  await fileRepository.create({
    originalName:
      file.originalname,

    storedName:
      storedFile.storedName,

    mimeType:
      file.mimetype,

    size:
      file.size,

    storageKey:
      storedFile.storageKey,

    uploadedById:
      userId,

    taskId,
  });
```

PART 40 — STORAGE FAILURE PROBLEM

Remember:

`Database transaction`

cannot rollback:

`fs.writeFile()`

Therefore:

`Physical storage`

and:

`Database`

need separate error handling.

PART 41 — SAFE UPLOAD

Use:

```
let storedFile;

try {

  storedFile =
    await storageService.save(file);

  const createdFile =
    await fileRepository.create({
      originalName:
        file.originalname,

      storedName:
        storedFile.storedName,

      mimeType:
        file.mimetype,

      size:
        file.size,

      storageKey:
        storedFile.storageKey,

      uploadedById:
        userId,

      taskId,
    });

  return createdFile;

} catch (error) {

  if (storedFile) {
    await storageService.delete(
      storedFile.storageKey,
    );
  }

  throw error;
}
```

This prevents orphan files when database creation fails.

PART 42 — ACTIVITY INTEGRATION

After successful database creation:

```
File created
```

```
    ↓
Activity created
```

Use your existing ActivityService.

Conceptually:

```
await activityService.createActivity({
  actorId: userId,
  action: "UPLOAD",
  targetType: "FILE",
  targetId: createdFile.id,
  taskId,
  description:
    `Uploaded ${file.originalname}`,
});
```

Adapt this to the exact input type you created for your Activity module.

PART 43 — CONTROLLER

Create:

```
src/modules/file/file.controller.ts
```

Import:

```
import type { Response } from "express";

import type {
  AuthenticatedRequest,
} from "../../types/auth.types.js";

import { FileService } from "./file.service.js";
```

Create:

```
const fileService =
  new FileService();
```

Then:

```
export const uploadTaskFile =
  async (
    req: AuthenticatedRequest,
    res: Response,
  ) => {

    const file = req.file;

    if (!file) {
      return res.status(400).json({
        status: "error",
        message: "File is required",
```

```
    });
  }

  const taskId =
    req.params.taskId;

  const userId =
    req.user.id;

  const result =
    await fileService.uploadTaskFile(
      file,
      userId,
      taskId,
    );

  return res.status(201).json({
    status: "success",
    data: result,
  });
};
```

Use your project's existing error-handling/`asyncHandler` pattern instead of adding a second error-handling style.

PART 44 — CONTROLLER RESPONSIBILITY

Controller should:

```
Receive HTTP request
        ↓
Extract data
        ↓
Call Service
        ↓
Return HTTP response
```

Controller should NOT:

```
❌ Call Prisma
❌ Save physical files
❌ Delete physical files
❌ Implement membership rules
❌ Implement project authorization
```

PART 45 — ROUTES

Create:

```
src/modules/file/file.routes.ts
```

Import:

```
import { Router } from "express";

import { isLogin }
  from "../../middleware/auth.middleware.js";

import { upload }
  from "../../config/multer.js";

import {
  uploadTaskFile,
} from "./file.controller.js";
```

Then:

```
const router =
  Router();

router.post(
  "/tasks/:taskId/files",
  isLogin,
  upload.single("file"),
  uploadTaskFile,
);

export default router;
```

PART 46 — MIDDLEWARE ORDER

The order is important:

```
isLogin
   ↓
upload.single("file")
   ↓
controller
```

Because:

```
isLogin
```

establishes:

```
req.user
```

and:

```
upload.single()
```

establishes:

```
req.file
```

before the controller executes.

PART 47 — REGISTER ROUTER

In your main app:

```
import fileRoutes
  from "./modules/file/file.routes.js";
```

Then:

```
app.use(
  "/api",
  fileRoutes,
);
```

Your endpoint becomes:

```
POST /api/tasks/:taskId/files
```

PART 48 — FIRST TEST

Use Postman.

Request

```
POST
/api/tasks/TASK_ID/files
```

Authentication:

```
Your existing authentication cookie
```

Body:

```
form-data
```

Field:

```
file
```

Type:

```
File
```

Select:

```
test.pdf
```

PART 49 — SUCCESS CONDITIONS

After uploading successfully, check PostgreSQL.

There should be a row in:

```
files
```

with:

```
originalName
storedName
mimeType
size
storageKey
uploadedById
taskId
```

Also check:

```
uploads/files/
```

You should see the generated file.

PART 50 — DO NOT MOVE ON UNTIL THIS WORKS

Your first milestone is:

```
LOGIN
 ↓
UPLOAD PDF
 ↓
MULTER
 ↓
VALIDATE
 ↓
AUTHORIZATION
 ↓
SAVE FILE
 ↓
SAVE DB RECORD
 ↓
ACTIVITY
 ↓
SUCCESS
```

Do not implement Project/Workspace/Comment file uploads until this works.

PART 51 — DOWNLOAD

After Task upload works, implement:

```
GET /api/files/:id/download
```

Architecture:

```
Controller
    ↓
FileService
    ↓
FileRepository
    ↓
Find metadata
    ↓
Authorization
    ↓
StorageService
    ↓
Physical file
    ↓
HTTP response
```

PART 52 — STORAGE `getPath()`

Update:

```
storage.types.ts
```

to eventually include:

```
getPath(
  storageKey: string,
): Promise<string>;
```

Then implement it in `LocalStorage`.

PART 53 — DOWNLOAD AUTHORIZATION

Never allow:

```
GET /files/some-id/download
```

to automatically return the file.

First:

```
Find File
    ↓
Find Task/Project/Workspace/Comment
    ↓
Check user permission
    ↓
Allow download
```

Authentication and authorization are different:

```
Authentication:
"Who are you?"

Authorization:
"Are you allowed to do this?"
```

PART 54 — DELETE

Implement:

```
DELETE /api/files/:id
```

Flow:

```
Authentication
      ↓
Find File
      ↓
Authorization
      ↓
Delete physical file
      ↓
Delete DB record
      ↓
Activity
      ↓
Response
```

PART 55 — DELETE ORDER

For our first local-storage implementation:

```
1. Find database record
2. Check authorization
3. Delete physical file
4. Delete database record
```

```
5. Create activity
```

If physical deletion fails, do not pretend the operation succeeded.

PART 56 — LIST TASK FILES

Endpoint:

```
GET /api/tasks/:taskId/files
```

Repository:

```
findByTask(taskId)
```

Service:

```
Check Task
Check authorization
Get files
Return files
```

PART 57 — LIST PROJECT FILES

Endpoint:

```
GET /api/projects/:projectId/files
```

Flow:

```
Authentication
 ↓
Authorization
 ↓
Repository
 ↓
Files
```

PART 58 — LIST WORKSPACE FILES

Endpoint:

```
GET /api/workspaces/:workspaceId/files
```

Check:

```
Is user a member of Workspace?
```

before returning files.

PART 59 — LIST COMMENT FILES

Endpoint:

```
GET /api/comments/:commentId/files
```

Check:

```
Comment
 ↓
Task
 ↓
Project
 ↓
Workspace
 ↓
User permission
```

PART 60 — IMPORTANT BUSINESS RULE

A File may have:

```
workspaceId
projectId
taskId
commentId
```

but these are optional because the same File module supports different contexts.

Examples:

Task attachment

```
taskId = TASK_ID
```

Project attachment

```
projectId = PROJECT_ID
```

Workspace attachment

```
workspaceId = WORKSPACE_ID
```

Comment attachment

```
commentId = COMMENT_ID
```

Do not blindly populate all four fields.

PART 61 — ACTIVITY ACTIONS

Your existing enum includes:

```
UPLOAD
DOWNLOAD
DELETE
```

Therefore create activities for:

```
UPLOAD
DOWNLOAD
DELETE
```

Example:

```
User uploaded report.pdf
```

Activity:

```
action = UPLOAD
targetType = FILE
targetId = file.id
actorId = user.id
```

PART 62 — FILE + ACTIVITY ARCHITECTURE

Do:

```
FileService
    │
    ├── FileRepository
    │
    ├── StorageService
    │
    └── ActivityService
```

Do NOT make:

```
FileRepository
    ↓
ActivityRepository
```

The Service coordinates business operations.

PART 63 — ERROR HANDLING

You already have your project error architecture.

Eventually replace:

```
throw new Error(
  "File not found"
);
```

with your existing:

```
AppError
```

and use:

```
asyncHandler
```

plus:

```
central error middleware
```

Do not create a completely different error system for File.

PART 64 — SECURITY REQUIREMENTS

Before calling File complete, verify:

```
[ ] Authentication required
[ ] Authorization required
[ ] File size limit
[ ] MIME type validation
[ ] Empty files rejected
[ ] Generated storage filename
[ ] Original filename only stored as metadata
[ ] No arbitrary filesystem path
[ ] Download authorization
[ ] Delete authorization
[ ] Orphan-file cleanup
```

PART 65 — DO NOT IMPLEMENT YET

Do not add these during the first offline implementation:



```
Supabase Storage
AWS S3
Cloudinary
File versioning
Public sharing links
```

- [ ] Virus scanning
- [ ] Image processing
- [ ] Video processing
- [ ] Chunked upload
- [ ] Resumable upload
- [ ] File preview generation
- [ ] Chat attachments

Those belong to the later refinement stage.

PART 66 — COMPLETE CODING CHECKLIST

Copy this section into Word and check each item.

DATABASE

[ ] 1. Add File model
[ ] 2. Add User → File relation
[ ] 3. Add Workspace → File relation
[ ] 4. Add Project → File relation
[ ] 5. Add Task → File relation
[ ] 6. Add Comment → File relation
[ ] 7. Run prisma format
[ ] 8. Run prisma validate
[ ] 9. Create migration
[ ] 10. Generate Prisma client

DEPENDENCIES

[ ] 11. Install multer
[ ] 12. Install @types/multer

TYPES

[ ] 13. Create file.types.ts
[ ] 14. Create CreateFileInput
[ ] 15. Create FileAttachmentInput

REPOSITORY

[ ] 16. Create file.repository.ts
[ ] 17. create()
[ ] 18. findById()
[ ] 19. findByTask()
[ ] 20. findByProject()
[ ] 21. findByWorkspace()
[ ] 22. findByComment()

[ ] 23. delete()

STORAGE

[ ] 24. Create storage.types.ts
[ ] 25. Create StoredFile
[ ] 26. Create StorageService
[ ] 27. Create local.storage.ts
[ ] 28. Implement save()
[ ] 29. Implement delete()
[ ] 30. Test storage directory

MULTER

[ ] 31. Create multer.ts
[ ] 32. memoryStorage()
[ ] 33. Configure file size
[ ] 34. Export upload

VALIDATION

[ ] 35. Create file.validation.ts
[ ] 36. Define allowed MIME types
[ ] 37. Validate MIME type
[ ] 38. Validate empty file
[ ] 39. Validate size

SERVICE

[ ] 40. Create file.service.ts
[ ] 41. uploadTaskFile()
[ ] 42. Validate file
[ ] 43. Check Task
[ ] 44. Check authorization
[ ] 45. Save physical file
[ ] 46. Create database record
[ ] 47. Cleanup physical file if DB fails
[ ] 48. Create Activity
[ ] 49. Return result

CONTROLLER

[ ] 50. Create file.controller.ts
[ ] 51. Extract req.file
[ ] 52. Extract taskId
[ ] 53. Extract req.user.id
[ ] 54. Call FileService
[ ] 55. Return HTTP response

ROUTES

```
[ ] 56. Create file.routes.ts
[ ] 57. Add authentication middleware
[ ] 58. Add Multer middleware
[ ] 59. Add controller
[ ] 60. Register routes
```

FIRST TEST

```
[ ] 61. Start server
[ ] 62. Login
[ ] 63. Create test Task
[ ] 64. Upload PDF
[ ] 65. Check DB
[ ] 66. Check uploads/files
[ ] 67. Check API response
```

DOWNLOAD

```
[ ] 68. Add storage getPath()
[ ] 69. Find file
[ ] 70. Check authorization
[ ] 71. Read/send file
[ ] 72. Create DOWNLOAD activity
[ ] 73. Test download
```

DELETE

```
[ ] 74. Find file
[ ] 75. Check authorization
[ ] 76. Delete physical file
[ ] 77. Delete DB record
[ ] 78. Create DELETE activity
[ ] 79. Test delete
```

LIST

```
[ ] 80. Task files
[ ] 81. Project files
[ ] 82. Workspace files
[ ] 83. Comment files
[ ] 84. Authorization for every list
```

PART 67 — THE ORDER YOU SHOULD ACTUALLY CODE

When you are offline, **do not jump around**.

Follow this exact sequence:

```
1. Prisma schema
        ↓
2. Migration
        ↓
3. file.types.ts
        ↓
4. file.repository.ts
        ↓
5. storage.types.ts
        ↓
6. local.storage.ts
        ↓
7. multer.ts
        ↓
8. file.validation.ts
        ↓
9. file.service.ts
        ↓
10. file.controller.ts
        ↓
11. file.routes.ts
        ↓
12. Register routes
        ↓
13. Upload Task file
        ↓
14. Test
        ↓
15. Download
        ↓
16. Test
        ↓
17. Delete
        ↓
18. Test
        ↓
19. Project files
        ↓
20. Workspace files
        ↓
21. Comment files
        ↓
22. Activity integration
        ↓
23. Final security/error cleanup
```

PART 68 — YOUR FIRST TARGET

Do **not** think:

> "I have to finish the entire File module."

Your first target is only:

```
Authenticated user
```

```
       ↓
Task
       ↓
Upload PDF
       ↓
Multer
       ↓
Validation
       ↓
Authorization
       ↓
LocalStorage
       ↓
PostgreSQL
       ↓
Activity
```

If that works correctly, you have the foundation.

Then:

```
Task
Project
Workspace
Comment
```

are mostly extensions of the same architecture.

PART 69 — ARCHITECTURE RULE TO REMEMBER

Keep this in your Word document:

```
CONTROLLER
"What HTTP request did I receive?"

SERVICE
"What business rule must happen?"

REPOSITORY
"How do I communicate with PostgreSQL?"

STORAGE SERVICE
"How do I save/read/delete the physical file?"

AUTHORIZATION
"Is this user allowed to perform this operation?"

ACTIVITY SERVICE
"What happened in the system?"
```

And the overall flow:

```
HTTP
 ↓
Route
 ↓
Middleware
 ↓
Controller
 ↓
Service
 ├── Repository → PostgreSQL
 ├── Storage → File system
 ├── Authorization
 └── ActivityService
```

That is the architecture I recommend you follow for the File module.

For your offline session, start only with Parts 5–23 (Prisma → Types → Repository), finish and test those first, then continue to Storage. This keeps the work manageable and makes it much easier to identify exactly where an error comes from.



